Introduction

Lino Galiana
Version of November 2, 2026

This introduction presents the course objective, pedagogical approach, the main theme of this of the course, as well as the practical practical details.

⚠️ **Automatically generated PDF.** This document is produced from the course website and may contain formatting issues. For an up-to-date version adapted to your reading, visit the course page: pythonds.linogaliana.fr.

Table of contents

1 Introduction **1**
2 Why `Python` ? **2**
3 Why use `Python` for data analysis? **3**
4 Why learn `Python` when code-generating AIs exist? **5**
4.1 Because coding means solving an engineering problem 6
4.2 Because coding means thinking 6
4.3 Because coding means constantly updating your knowledge 7
5 Course Objectives **7**
5.1 Introducing the data science approach 7
5.2 Reproducibility 8
5.3 Assessment 9
6 Course Outline **9**
References **13**
Informations additionnelles 13
Bibliography **15**

Skills you will acquire in this chapter

- Understand why Python has become essential in the fields of *data science* and *data engineering*, thanks to its simplicity, readability, and strong community ecosystem
- Recognize how Python can support you through every stage of a data-driven project—from structuring and exploring data to modeling and communicating results
- Appreciate the value of a reproducible, rigorous, and scientific approach to *data science* and *data engineering*, and learn how to support this using tools like Jupyter notebooks, open data, and standardized environments
- Grasp the purpose and teaching goals of the course, as well as its main themes

1 Introduction

This course brings together all the material from the ***Python for Data Science*** class I've been teaching at ENSAE since 2020, and at ENSAI since 2026. More than 300 students now take this course each year.

> **International version**
>
> In 2024, an English version—equivalent to the French original—was gradually introduced. It is designed as an introductory *data science* course for European statistical institutes, following a European call for projects.

The site (pythonds.linogaliana.fr) serves as the main hub for the course. It gathers all course content, including practical assignments and additional materials aimed at continuing education. The course is *open source*, and I welcome suggestions for improvement either on `GitHub` or in the comments section at the bottom of each page.

Starting with the 2026 academic year, this course undergoes a major change: it is now split into two parts, taught across two semesters, allowing each to be covered in greater depth:

- Python for *data* in the first semester, corresponding to the first part of the course focused on data exploration;
- Python for *ML* in the second semester, corresponding to the second part of the course focused on modeling.

Because `Python` is a living, fast-evolving language, the course is continuously updated to reflect the changing *data science* ecosystem. At the same time, it strives to differentiate lasting trends from short-lived fads.

While this course is primarily aimed at *data science* students, it is general enough to be of interest to a wide range of profiles—students and professionals alike—coming from diverse academic backgrounds. It is designed as an introduction to *data science* through Python.

> **Going further**
>
> You can find more information in the introductory slides. More advanced topics are covered in another course focused on deploying *data science* projects to production, which I co-teach with Romain Avouac in the final year at ENSAE (ensae-reproductibilite.github.io/website), with a variant of it soon to be taught at ENSAI as well.

> **Website Architecture**
>
> This course offers comprehensive tutorials and exercises that can be read directly on the site or edited and run in an interactive `Jupyter Notebook` environment (see the next chapter for details).
>
> Each page is built around a concrete problem and introduces a general approach to solving it. All examples are based on *open data* and are fully reproducible.
>
> You can navigate the site using the table of contents or the previous/next links at the bottom of each page. Some sections - such as the one on modeling - include highlighted examples that illustrate the methodology and present different possible approaches to solving the same problem.

2 Why `Python` ?

`Python` whose recognizable logo appears as , is a language that's been around for over thirty years. But it was in the 2010s that it experienced a major resurgence, driven by the growing popularity of *data science*.

More than any other language, `Python` brings together a wide range of communities: statisticians, application developers, IT infrastructure managers, high school students (it has been part of the French baccalaureate curriculum for several years), and researchers in both theoretical and applied fields.

Unlike many programming languages that cater to relatively homogeneous communities, `Python` has succeeded in uniting diverse users thanks to a few key principles: its readable syntax, the simplicity of using modules, the ease of integration with more powerful languages for specific tasks, and the vast amount of online documentation. Sometimes, being the second-best tool for a task—while offering a broader set of advantages—can be the key to success.

`Python` success story is closely tied to the rise of the *data scientist* role, a profile capable of working across the entire data processing pipeline. In the *Harvard Business Review*, T. H. Davenport and D. Patil [1] famously called it "the sexiest job of the 21st century." A decade later, he and his co-authors provided a full update on the evolving expectations for *data scientists* [2].

But it's not only *data scientists* who need to use `Python`. In the broader ecosystem of data-related professions—*data scientists*, *data engineers*, *ML engineers*, and more—`Python` serves as a kind of Tower of Babel, enabling collaboration among interdependent roles[1].

This course introduces various tools that use `Python` to connect data with theoretical concepts from statistics and the economic and social sciences. However, it goes beyond a simple introduction to the language: it regularly reflects on both the strengths and the limitations of `Python` in meeting operational and scientific needs.

3 Why use `Python` for data analysis?

This question is slightly different: if `Python` is already a popular language for learning programming due to its ease of use, how did it also become the dominant language in the *data* and AI ecosystem?

Python first gained traction in the *data science* world for offering tools to train *machine learning* algorithms, even before such approaches became mainstream. Of course, the success of libraries like `Scikit-Learn`, `TensorFlow`, and more recently `PyTorch`, played a major role in `Python`'s adoption by the *data science* community[2]. However, reducing `Python` to a handful of *machine learning* libraries would be overly simplistic. It is truly a Swiss Army knife for *data scientists*, social scientists, economists, and data practitioners of all kinds. Its success is not only due to offering the right tools at the right time, but also because the language itself offers real advantages for newcomers to data work.

What makes `Python` appealing is its central role in a broader ecosystem of powerful, flexible, and open-source tools. Like , it belongs to a category of languages suitable for everyday use across a wide variety of tasks. In many of the fields covered in this course, `Python` has by far the richest and most accessible ecosystem. Unlike other popular languages such as `JavaScript` or `Rust`, it has a very gentle learning curve, allowing users to write high-quality code quickly - provided they learn the right habits, which this course (and the companion course on production workflows) aims to instill.

[1] See L. Galiana and O. Lefebvre [3] for more context on the evolving landscape of data-related jobs.

[2] `Scikit-Learn` is a library developed since 2007 by French public research labs at INRIA. Open source from the outset, it is now maintained by `:probabl.`, a startup created to manage the `Scikit` ecosystem, bringing together some of the INRIA research teams behind the core of the modern *machine learning* stack.

`TensorFlow` was developed internally at Google and made public in 2015. Although now less widely used - partly due to the rise of `PyTorch` - it played a major role in popularizing neural networks in both research and production during the 2010s.

`PyTorch` was developed by Meta starting 2018 and has been governed by the *PyTorch Foundation* since 2022. It is now the most widely used framework to train neural networks.

Beyond AI projects[^nte-ia-en], `Python` is also indispensable for retrieving data via APIs or through *web scraping*[3], two techniques introduced early in the course. In areas like tabular data analysis[4], web publishing or data visualization, `Python` now offers an ecosystem comparable to , thanks in part to growing investment from `Posit`, which has ported many of ‘s most successful libraries—such as ggplot to `Python`[5].

> **Tabular data** is structured data, organized — as the name suggests — in table form, mapping observations to variables. This structure contrasts with other, more complex data types: free text, images, audio, video…

ℹ Note 1: Why discuss AI so little in a Python course?

[3] In the domains of API access and *web scraping*, `JavaScript` is `Python`’s most serious competitor. However, its community is more focused on web development than on *data science*.

[4] In the field of unstructured data, `Python` is the dominant analysis language. For tabular data, `Python`’s competitive edge is smaller, particularly compared to , but the two languages offer a fairly similar core set of features. We will regularly draw parallels between them in the chapters devoted to the `Pandas` library.

That said, this course does not aim to stir up the sterile debate between and `Python`.

The two languages share far more than they differ, and best practices are easily transferable from one to the other. This idea is explored in more depth in the more advanced course I co-teach with Romain Avouac in the final year at ENSAE: ensae-reproductibilite.github.io/website.

[5] In practice, data scientists and researchers in the social sciences or economics increasingly use and `Python` almost interchangeably, switching between the two. This course will therefore regularly draw analogies with to help those discovering `Python`, but already familiar with , better understand certain points.

While a significant portion of this course covers *machine learning* and related algorithms, I tend to resist the current trend - especially strong since the release of `ChatGPT` in late 2022 - of labeling everything as "AI".

First, because the term is vague, overused, and often exploited for marketing purposes, capitalizing on its symbolic power drawn from science fiction to sell miracle solutions or stoke fear.

Second, because the term "AI" covers a vast range of possible methods, depending on how broadly we define it. The sections on modeling and NLP in this course, which are the closest to the AI field, focus on learning-based methods. But as definitions from S. J. Russell and P. Norvig [4] or the European AI Act show, artificial intelligence encompasses much more:

> The study of [intelligent] agents that perceive their environment and act upon it. Each such agent is implemented by a function that maps perceptions to actions. We study different ways to define this function, such as production systems, reactive agents, logical planners, neural networks, and decision-theoretic systems.
>
> S. J. Russell and P. Norvig [4]

> "AI system" means a machine-based system designed to operate with varying levels of autonomy and capable of adapting after deployment. It infers, based on its inputs, how to generate outputs—such as predictions, content, recommendations, or decisions—that can influence physical or virtual environments.
>
> European AI Act

For more on this topic, see a presentation I gave in 2024 (in French):

Finally, there's also a pedagogical reason. Since 2023, "AI" has largely become synonymous with generative AI. But to understand how this radically different paradigm works - and to implement meaningful, value-driven generative AI projects - one must first understand the foundations and limitations of the *machine learning* approach. Otherwise, there's a risk of building overly complex solutions for simple problems or misjudging the value of generative models compared to more traditional methods. Since this is an introductory course, I've chosen to focus on *machine learning* and introductory NLP, deep enough to be meaningful, while leaving it to the curious to explore generative AI further on their own.

4 Why learn `Python` when code-generating AIs exist?

Code assistants, especially `Claude` and `ChatGPT`, have changed the way we write code forever. The pace of change is quite staggering.

These tools are now part of the everyday toolkit of *data scientists* and offer real convenience, since they can generate `Python` code from instructions that are more or less clearly specified. And because these AIs have been trained on virtually all the code available on the internet—and are sometimes specialized for development tasks—they help a great deal.

The integration of *vibe coding* tools into IDEs (particularly `Claude Code`) has accelerated this process even further and poses a real challenge for education.

So, if AIs can now generate code, why should we still learn how to code?

4.1 Because coding means solving an engineering problem

Because coding is not just about writing lines of code. It's about understanding a problem, crafting a step-by-step strategy to tackle it, considering multiple solutions and trade-offs (e.g., speed, simplicity), testing and debugging. Code is a means to an engineering end. While AIs are very good at generating code, relating problems to known patterns, and even translating solutions across languages into `Python`, that's only part of the picture.

But you still need to know how to formulate the problem, how to judge the quality of the LLM's answer, and how to challenge the assistant's suggestion in order to fix a mistake or get a more satisfying answer. LLMs are a highly (very highly) refined version of a *Google* search: if you don't have the right keywords for a Google search, your search will be disappointing. The same goes for LLMs, even though the conversational, natural-language aspect lowers the barrier to entry.

Code is not an end in itself, but a tool in service of structured reasoning aimed at solving a concrete problem.

Working with a dataset, and searching for answers that aren't obvious, raises a variable mix of scientific and engineering questions. Like an engineer designing a bridge, who works to meet the need to cross a gap using the scientific knowledge and materials available, the *data scientist* starts from an operational goal (building a recommendation algorithm, measuring the impact of a product launch, forecasting sales) and turns it into something workable. This means translating scientific or *business* concepts into analytical questions, then breaking the problem down into logical steps, each of which is translated into instructions the machine can execute.

In this context, an LLM can act as an assistant, but only if the problem is well posed. If the steps are vague or poorly defined, the model's answer will be approximate, or even useless. The frontier keeps being pushed back thanks to models' growing reasoning capabilities, but these tools remain limited. On a standard task, the result may look correct, but on a more specific question, it will often be necessary to refine, rephrase, iterate… and sometimes never get a satisfactory answer. Not because the model is bad, but because upstream problem engineering makes all the difference[6].

4.2 Because coding means thinking

What's more, if we don't take the time, during our studies, to grapple with these problems and to learn, when will we ever have the time to do so? Although we still have little hindsight on this topic, cognitive science research is starting to accumulate evidence of a negative effect of AI use on the brain (see the concept of "cognitive debt" in N. Kosmyna *et al.* [5]).

This isn't about being nostalgic for the golden days when you could spend hours racking your brain over a coding problem that an AI can now solve in minutes, or even seconds. I myself use generative AI every day to help generate code or create content (for instance, for the automatic translation of this course into English). That said, it's worth remembering that the goal is to enable more *deep work* (and to enjoy it!), not to end up as a mere alienated machine, condemned to clicking the *"Yes"* button as fast as possible whenever Claude asks for permission. This is a hard balance to strike, since the tools available today tend to make us dependent on them. The best way to avoid becoming captive to these tools remains acquiring—and continuing to build throughout one's career—general skills: that is the ambition of this applied *data science* course.

[6] On this topic, see Thomas Wolf's blog post *The Einstein AI model*. Although the post focuses on disruptive innovation and pays less attention to incremental progress, it's insightful in understanding that LLMs—despite bold predictions from tech influencers—are still just tools. They may excel at standardized tasks, but for now, they remain assistants.

4.3 Because coding means constantly updating your knowledge

One last reason not to rely on a code-generating AI without critical distance is that these tools are inevitably behind current practice, since they were trained on past data. The `Python` ecosystem is highly dynamic, and even though the AIs from major providers are frequently retrained and can now access the internet to refresh their knowledge, certain libraries can quickly establish themselves in their field.

For instance, last year `uv` saw rapid adoption, as did `ruff` the year before. The existence of generative AIs does not, therefore, exempt us, as before, from keeping an active technical watch and being vigilant about changes in practices.

5 Course Objectives

5.1 Introducing the data science approach

This course is intended for practitioners of *data science*, understood here in the broadest sense as the **combination of techniques from mathematics, statistics, and computer science to extract useful knowledge from data**.

Since *data science* is not only an academic discipline but also a practical field aimed at achieving operational goals, learning its main tool—namely, the `Python` programming language—goes hand in hand with adopting a rigorous, scientific approach to data.

The objective of this course is to explore how to approach a dataset, identify and address common challenges, develop appropriate solutions, and reflect on their broader implications. It is therefore not merely a course about a technical tool, disconnected from scientific reasoning, but one rooted in understanding data through both technical and conceptual lenses.

Do I need a math background for this course?

This course assumes you are interested in using data-intensive `Python` within a rigorous statistical framework. It does not delve deeply into the statistical or algorithmic foundations of the techniques presented - many of which are covered in dedicated courses, particularly at ENSAE.

That said, not being familiar with these concepts shoud not prevent from following this course. More advanced ideas are typically introduced separately, in dedicated callout boxes. Thanks to Python's ease of use, you will not need to implement complex models from scratch - making it possible to apply techniques even if you are not an expert in the underlying theory. What *is* important, however, is having enough understanding to correctly interpret the results.

Still, while `Python` makes it relatively easy to run sophisticated models, it is very helpful to have some perspective before diving into modeling. That explains why modeling appears later in the course: in addition to relying on advanced statistical concepts, effective modeling also requires a solid understanding of the data. You need to identify key patterns and assess whether your data fits the assumptions of the model. Without this foundation, it is difficult to build models that are truly meaningful or reliable.

5.2 Reproducibility

This course places strong emphasis on reproducibility. This principle is reflected in several ways. First and foremost, by ensuring that all examples and exercises can be run and tested using `Jupyter` *notebooks*.

> A *notebook* is an interactive environment for writing and running code live. It combines, in a single document, text and code that can be executed, with outputs displayed right after computation. It's extremely convenient for learning the `Python` language.
>
> For more details, see the official Jupyter documentation.

All content on the website is designed to be reproducible across different computing environments. Of course, you're free to copy and paste code snippets directly from the site using the button available at the top of each code block.

```
x = "Try to copy-paste me"
```

Line 1
Click on the button to copy this content and paste it elsewhere.

However, since this site includes many examples, constantly switching between a `Python` environment and the website can become tedious. To make things easier, each chapter can be downloaded as a `Jupyter` *notebook* using the buttons provided at the top of each page.

For example, here are the buttons for the first chapter on `Pandas`:

Recommendations on the best environments for using these notebooks are provided in the next chapter.

The focus on reproducibility is also reflected in the choice of examples used throughout the course. All content on this site is based on *open data*, sourced either from French platforms - primarily the centralized portal `data.gouv`, which aggregates public datasets from French institutions, or the official statistics agency Insee, France's national institute for statistics and economic studies - or from U.S. datasets. This ensures that results are reproducible for anyone working in an identical environment[7].

> **i** Note
>
> American researchers have described a reproducibility crisis in the field of *machine learning* [6]. The distortions of the scientific publishing ecosystem - combined with the economic incentives driving academic publications in *machine learning* - are often cited as major contributing factors.
>
> However, university education also bears a share of the responsibility. Students and researchers are rarely trained in the principles of reproducibility, and if these practices are not introduced early in their careers, they are unlikely to adopt them later. This is why, in addition to teaching `Python` and *data science*, this course includes a dedicated section on using version control with `Git`.
>
> All student projects are required to be *open source*—one of the most effective ways for instructors to encourage high-quality, transparent, and reproducible code.

[7] Opening the chapters as *notebooks* in standardized environments - something explained in the next chapter - ensures you are working in a controlled setup. Personal `Python` installations often involve tweaks and adjustments that can alter your environment and lead to unexpected, hard-to-diagnose errors. For this reason, such local setups are not recommended for this course. As you'll see in the next chapter, *cloud-based* environments offer the advantage of consistent, preconfigured setups that greatly improve reliability and ease of use.

5.3 Assessment

Students at ENSAE and ENSAI complete the course by working on an in-depth project. Details on how the course is assessed, along with a list of past student projects, can be found in the Evaluation section.

6 Course Outline

This course serves as an introduction to the core challenges of *data science* through learning the `Python` programming language. As the term *"data science"* implies, a significant portion of the course is dedicated to working directly with data: retrieving it, structuring it, exploring it, and combining it.

These topics are covered in the first part of the course, "Manipulating Data", which lays the foundation for everything that follows. Unfortunately, many training programs in *data science*, applied statistics, or the economic and social sciences tend to overlook this crucial aspect of a data scientist's work—often referred to as "data wrangling" or *"feature engineering"*. And yet, not only does it represent a large share of the day-to-day work in data science, it's also essential for building relevant and accurate models.

The goal of this first section is to highlight the challenges involved in accessing and leveraging different types of data sources using `Python`. The examples are diverse, reflecting the variety of data that can be analyzed with `Python`: municipal CO_2 emissions in France, real estate transaction records, housing energy performance diagnostics, bike-sharing data from the Velib system, and more.

The second part of the course focuses on creating visualizations with `Python`. Once your data has been cleaned and processed, you'll typically want to summarize it—through tables, graphs, or maps. This part, "Communicating with Python", offers a concise introduction to the topic. While somewhat introductory, it provides essential concepts that will be reinforced later in the course.

The third part centers on modeling, using electoral science as the main example ("Modeling with Python"). This section introduces the scientific reasoning behind *machine learning*, explores both methodological and technical choices, and sets the stage for deeper topics addressed later in the program.

The fourth part takes a step back to focus on the specific challenges of working with text data. This is the "Introduction to Natural Language Processing (NLP) with Python" chapter. Given that NLP is a rapidly evolving field, this section serves only as an introduction. For more advanced coverage, see S. J. Russell and P. Norvig [4], chapter 24.

This chapter also includes a section on version control with `Git` (Discover `Git`). Why include this in a course about `Python` ? Because learning `Git` helps you write better code, collaborate effectively, and test or share your work in reproducible environments. This is especially valuable in a world where platforms like `GitHub` act as professional showcases—and where companies and public institutions increasingly expect their *data scientists* to be proficient with `Git`.

For more advanced applications, including deployment and reproducibility, refer to the companion course on putting data science projects into production.

In more detail, here is the course outline:

Introduction

Title	Available in	Learning mode	Resource type
Introduction	France United States	Lesson #1 or self learning	Read from website
Avoir un environnement Python fonctionnel pour la data science	France United States	Lesson #1 or self learning	Read from website and exercise notebooks
Démarche à adopter face à un jeu de données	France United States	Lesson #1 or self learning	Read from website
Quelques exercices de rappels pour se remettre en selle	France United States	Lesson #1 or self learning	Read from website

Manipuler des données

Title	Available in	Learning mode	Resource type
Introduction à la partie 1: manipuler des données	France United States	Lesson #1 or self learning	Read from website
Numpy, la brique de base de la data science	France United States	Lesson #1 or self learning	Read from website and exercise notebooks
Introduction à Pandas	France United States	Lesson #2 or self learning	Read from website and exercise notebooks
Associer des données avec Pandas : les jointures	France United States	Lesson #2 or self learning	Read from website and exercise notebooks
Manipuler des données avec Pandas : statistiques et restructuration	France United States	Lesson #2 or self learning	Read from website and exercise notebooks
Pandas : limites et écosystèmes alternatifs	France United States	Lesson #2 or self learning	Read from website
Introduction aux données spatiales avec Geopandas	France United States	Lesson #4 or self learning	Read from website and exercise notebooks

Title	Available in	Learning mode	Resource type
Web scraping avec Python	France United States	Self learning	Read from website and exercise notebooks
Récupérer des données avec des API depuis Python	France United States	Lesson #4 or self learning	Read from website and exercise notebooks
Maîtriser les expressions régulières	France United States	Self learning	Read from website and exercise notebooks
Les nouveaux modes d'accès aux données : le format Parquet et les données sur le cloud	France United States	Self learning	Read from website

Communiquer

Title	Available in	Learning mode	Resource type
Introduction à la partie 2: communiquer à partir de données	France United States	Self learning	Read from website
Construire des graphiques avec Python	France United States	Self learning	Read from website and exercise notebooks
Introduction à la cartographie avec Python	France United States	Self learning	Read from website and exercise notebooks

Modéliser

Title	Available in	Learning mode	Resource type
Introduction à la partie 3: modéliser	France United States	Lesson #5 or self learning	Read from website
Préparation des données pour construire un modèle	France United States	Lesson #5 or self learning	Read from website and exercise notebooks

Title	Available in	Learning mode	Resource type
Evaluer la qualité d'un modèle	France United States	Lesson #5 or self learning	Read from website and exercise notebooks
Découverte de la classification avec les arbres de décision	France United States	Lesson #5 or self learning	Read from website and exercise notebooks
Introduction à la régression	France United States	Lesson #6 or self learning	Read from website and exercise notebooks
Sélection de variables : une introduction	France United States	Lesson #6 or self learning	Read from website and exercise notebooks
Introduction à l'apprentissage non supervisé avec le clustering	France United States	Lesson #6 or self learning	Read from website and exercise notebooks
Premier pas vers l'industrialisation avec les pipelines scikit	France	Self learning	Read from website and exercise notebooks
Mettre à disposition un modèle par le biais d'une API	France	Self learning	Read from website

Natural Language Processing (NLP)

Title	Available in	Learning mode	Resource type
Introduction à la partie 4 : Natural Language Processing (NLP)	France United States	Self learning	Read from website
Nettoyer et structurer l'information dans les données textuelles	France United States	Lesson #7 or self learning	Read from website and exercise notebooks
L'analyse fréquentiste par l'approche bag-of-words : intérêt et limites	France United States	Lesson #7 or self learning	Read from website and exercise notebooks
Les embeddings, ou comment synthétiser l'information textuelle	France United States	Lesson #7 or self learning	Read from website and exercise notebooks

Git

Title	Available in	Learning mode	Resource type
Git: un outil nécessaire pour les data scientists	France United States	Lesson #3 or self learning	Read from website
Découvrir Git par la pratique: la gymnastique quotidienne	France United States	Lesson #3 or self learning	Read from website
Un cadavre exquis pour découvrir le travail collaboratif `Git`	France United States	Lesson #3 or self learning	Read from website

References

Informations additionnelles

ℹ Python environment

This site was built automatically through a `Github` action using the `Quarto` reproducible publishing software (version 1.10.18).

The environment used to obtain the results is reproducible via uv. The `pyproject.toml` file used to build this environment is available on the `linogaliana/python-datascientist` repository

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
```

```
    "loguru==0.7.3",
    "markdown>=3.8",
    "nbclient>=0.10.0",
    "nbformat>=5.10.4",
    "nltk>=3.9.1",
    "openpyxl>=3.1.5",
    "pandas>=3.0",
    "pip>=25.1.1",
    "plotly>=6.1.2",
    "plotnine>=0.15",
    "polars>=1.8.2",
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

To use exactly the same environment (version of `Python` and *packages*), please refer to the documentation for `uv`.

Bibliography

[1] T. H. Davenport and D. Patil, “Data scientist, the sexiest job of the 21st century,” *Harvard business review*, vol. 90, no. 5, pp. 70–76, 2012, [Online]. Available: https://hbr.org/2012/10/data-scientist-the-sexiest-job-of-the-21st-century

[2] T. H. Davenport and D. Patil, “Is data scientist still the sexiest job of the 21st century?,” *Harvard Business Review*, vol. 90, 2022.

[3] L. Galiana and O. Lefebvre, “Qu'apportent les métiers de la data à la statistique publique ?,” *Courrier des Statistiques*, no. 15, pp. 95–116, 2026, [Online]. Available: https://insee.hal.science/hal-05684911v1

[4] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach (4th Edition)*. Pearson, 2020. [Online]. Available: http://aima.cs.berkeley.edu/

[5] N. Kosmyna *et al.*, “Your brain on ChatGPT: Accumulation of cognitive debt when using an AI assistant for essay writing task,” *arXiv preprint arXiv:2506.08872*, vol. 4, p. 206, 2025.

[6] S. Kapoor and A. Narayanan, “Leakage and the Reproducibility Crisis in ML-based Science.” [Online]. Available: https://arxiv.org/abs/2207.07048